Maximum Semantic Capacity via the Ontology of Optimal Information Structure: A Unified Framework Across Visual Encoding and Agent Calibration

Ryan St Dare

November 5, 2026

Abstract

Information systems store, send, and reason over meaning at very different densities. A knowledge graph may need thousands of bytes to hold a few hundred facts. A language model may need thousands of words to hold a comparable set of facts. A software agent may need a long prompt to keep track of one decision. This paper argues that these are not separate problems. They are one problem. We call it **semantic capacity**: how much real meaning fits into each byte of data.

The main cause of low semantic capacity is not a shortage of storage or speed. It is the lack of a clear *structure* for meaning. Most formats, such as JSON and Markdown, were designed to be readable by people, not to pack meaning tightly. They repeat labels, punctuation, and instructions in every record. A small part of each record carries new meaning, and most of the bytes do not.

We introduce **ONE** (Ontology of Optimal Information Structure), a unified framework that explains how information can be structured so that meaning is stored densely, checked for damage, and used by both people and machines. ONE has three parts. First, it defines a simple shared grammar called **O-OIS**. This grammar says what kinds of things can appear in information, what kinds of connections can join them, how certain each connection is, and when information should be loaded or stored. Second, it shows the same grammar in three different forms: a fixed-size visual code called **POINTS**, a large visual symbol collection called **65536**, and a small work-memory system for software agents called **MSCA**. Third, it defines a simple measure, **semantic capacity**, that counts how many bits of meaning fit in each byte.

Across the POINTS test suite (94 tests plus 42 subtests) and a deliberate damage test, ONE achieves **9.7**× the semantic density of compact JSON when a visual code is close to full. At that point, POINTS holds 460 entities in 8 KB while JSON needs about 77.7 KB for the same facts. The code can repair damage: Reed–Solomon parity restored every tested case with up to 32 damaged bytes. The agent memory system keeps its always-active rules to 621 tokens, well below a 640-token budget, and all 15 safety and quality checks pass. We release all code, test data, glyphs, and numbers so that others can repeat every measurement.

Keywords: ontology of information, semantic capacity, knowledge-graph encoding, visual glyph, Reed-Solomon, agent calibration, tiered context loading, reproducibility, POINTS, MSCA, 65536

1 Introduction

1.1 The problem in plain language

Imagine that you must send a list of facts to a friend on a small card. You have two ways to do it. In the first way, you write every fact in long sentences, with repeated labels and punctuation. In the second way, you agree on a short code first: each thing gets a number, each kind of connection gets a sign, and the card holds only the numbers and signs. The second card holds far more facts.

That is the difference this paper studies.

Modern information systems mostly use the first way. JSON, for example, is useful because people can read it. But for every fact it repeats field names, brackets, commas, and quotes. The Software agent prompts used by language models do something similar: they repeat general instructions and reminders in many words, even though only a few words change from one decision to the next. The result is **semantic sparsity**: most of the stored data does not add new meaning.

The question is simple. *How much real meaning can we fit into one byte of data, and what structure makes that possible?*

1.2 What this paper does

This paper introduces **ONE**, a shared way to describe information so that meaning is packed densely. ONE is not one program. It is one idea shown in three forms, plus a way to measure the gain.

ONE makes three contributions:

1. **A shared grammar, called O-OIS.** O-OIS is a small set of rules that say: what things (entities) may appear, how things may be connected (relations), how parts may belong to a whole (meronymy), how sure we are about each connection (epistemic tags), and when information should be kept in short-term memory or moved to storage (loading controls). Appendix A gives the full formal rules in a standard notation called EBNF. Section 3 explains every part in plain language.

2. **The same grammar in three forms.** Section 4 explains each form, what it does, why it was chosen, and how it was built:

   - **POINTS** is a fixed-size visual code. A whole knowledge graph is packed into exactly 8,192 bytes, shown with raised-dot patterns like Braille. It can be read by a person, by a camera, or by a computer, and it can repair damage.
   - **65536** is a large collection of symbols, like a font with 65,536 meaningful pictures. Each picture stands for one idea, category, or connection. It shows that the same grammar can work as a visual language.
   - **MSCA** is a small memory discipline for software agents. It keeps only the most important rules always active (621 tokens, within a 640-token budget) and loads the rest only when needed. It shows that the same grammar can make agent memory smaller and more honest.

3. **A simple measure, called semantic capacity (SC).** SC counts how many bits of meaning fit in each byte. Section 5 defines it carefully, explains what it counts, and states clearly what it does not count. Section 6 reports the measurements.

We test ONE on a fixed example set of seven small graphs, on graphs that grow from 1 to 460 entities, on a rich graph with 300 entities plus extra details, and on deliberately damaged codes. The main results are:

- **Crossover.** For very small graphs, JSON is more compact because POINTS always uses a full 8,192 bytes. Once a graph has about 50 entities, POINTS becomes denser. At 460 entities, POINTS holds 4.48 bits of meaning per byte and JSON holds 0.46—a **9.7**× advantage (Table 4).
- **Rich graphs.** With 300 entities, 299 connections, 30 facets, and 30 salience notes, POINTS holds 2.99 bits per byte and JSON holds 0.45—a **6.6**× advantage (Table 5).
- **Damage repair.** Reed–Solomon parity restored every tested case with up to 32 damaged bytes (Table 6).
- **Tests.** POINTS: 94 tests plus 42 subtests pass. MSCA: 15 of 15 checks pass. In each case, tests were also broken on purpose to prove they would notice damage.

1.3 What "optimal" means here

The title uses the word *optimal*. We must explain what that word means, because it can be misunderstood.

In this paper, *optimal* does not mean "proved perfect for all time." No human-made code can prove that. It means: *the best carefully characterised structure we know for this stated purpose, with the stated limits, as of the stated date.* The purpose is to maximise meaning per byte while keeping the code fixed-size, repairable, and understandable to people. The limits are stated in Section 9. This honest use of the word follows philosophers such as Kant, who called such a target a *regulative ideal*: a direction to aim for, not a claim of arrival.

1.4 What this paper does not do

A second project, called TWO, concerns whether the Earth can be made fully sustainable through carbon, biochar, and related materials processes. That project is important, but it is a different question about the physical world. It is not part of this paper, and it is mentioned only to avoid confusion.

Paper Structure

Section 2 explains how ONE differs from existing formats and systems. Section 3 teaches the shared grammar, part by part, with examples. Section 4 explains the three forms and why each design choice was made. Section 5 defines semantic capacity and its limits. Section 6 reports what was measured and how to read the tables. Section 7 explains the agent memory study. Section 8 explains how every reported number was verified. Section 9 states every important limit in plain language. Section 10 summarises what was learned.

2 Related Work

ONE stands at the meeting point of four older lines of work. This section explains each line in plain language and shows where ONE is new.

2.1 Knowledge-graph encoding: how facts are stored on disk

A knowledge graph is a list of things and the connections between them. For example: "Sparrow *is a* Bird" and "Bird *can fly*."

Common text formats [3, 13]—RDF/XML, Turtle, JSON-LD, and N-Triples—were designed so that people and web tools can read them easily. That readability costs space: labels and punctuation are repeated for every fact.

Compact binary formats [7, 10]—Protocol Buffers, MessagePack, CBOR, and FlatBuffers—save space. But they do not carry the extra information ONE needs: how certain is each connection, is it affirmed or denied, how important is it, and can damage be repaired?

Graph-specific libraries such as GraphBLAS and WebGraph are optimised for fast traversal (following connections quickly), not for packing meaning into few bytes.

POINTS is different in four ways. It uses a fixed size. It can repair damage. A person can check it by eye. And every connection carries its own certainty and importance. The rest of this paper explains how.

2.2 Ontology design: how kinds of things are described

An ontology is a careful list of the kinds of things in a domain and how they relate. Large philosophical ontologies such as BFO, DOLCE, and SUMO are precise, but they are hard to use in code and they were not designed for dense storage.

Smaller web ontologies such as SKOS, Schema.org, and OWL-RL are easier to use, but they give up some precision to stay manageable.

O-OIS takes a different path. It is a *working* ontology. Every idea in it—every kind of connection, every way a part can belong to a whole, every certainty mark—maps directly to a small field of bits in the POINTS code. The ontology is therefore not only a description. It is also the code itself. Section 3 teaches each idea with a short example.

2.3 Agent context management: how software agents remember

A software agent that uses a language model receives a "context" of words on every step. Common frameworks [2, 8, 15]—LangChain, AutoGPT, ReAct, Voyager, and Generative Agents—treat this context as one long, flat window. When it grows too large, they summarise old parts, search for relevant parts, or compress the text. These methods help, but they are hard to measure and hard to test.

MSCA treats memory in two layers. A tiny set of rules is always present. Everything else is loaded only when the current phase needs it, and removed when it does not. The size of the always-present layer has a hard budget (640 tokens). Whether a rule set is kept or loaded is decided by an explicit score, not by hope. Section 7 explains this in detail and reports the measurements.

2.4 Visual encoding: information you can see

QR codes [14], DataMatrix codes [9], and Aztec codes store data in pictures. Braille [1] stores language in raised dots. These systems show that a picture can carry data and survive copying as an image.

POINTS uses Braille-like dots for a different reason. The dots make the code *checkable by people*. A printed glyph can be examined by eye, photographed, or read by a computer. The 65536 collection generalises the idea: instead of dots, each meaning has its own picture, like a very large alphabet where every symbol is a whole idea.

2.5 Semantic density measures: counting meaning, not just bytes

Information theory gives us Shannon entropy [12] (how surprising a message is) and Kolmogorov complexity (how short a program can make it). Both measure compressibility, not meaning. A file can compress well and still say little.

No earlier work we are aware of defines a simple cross-format measure of *bits of meaning per byte*. Section 5 defines semantic capacity (SC) for that purpose. It is deliberately simple: count the atomic facts that a competent reader can recover from the representation alone, assign each a conservative 10 bits, and divide by the number of bytes. The section states exactly what this number can and cannot tell us.

2.6 Summary: where ONE is new

Table 1: Positioning of ONE compared with related systems. A tick means the system provides that feature.

System	Fixed size	Repair	Certainty tags	Tiered memory	SC measure
JSON / MessagePack	✗	✗	✗	✗	✗
Protocol Buffers	✗	✗	✗	✗	✗
RDF / JSON-LD	✗	✗	✓ (limited)	✗	✗
LangChain / AutoGPT	✗	✗	✗	✗ (heuristic)	✗
QR / DataMatrix	✓	✓ (RS)	✗	✗	✗
ONE (this work)	✓	✓ (RS/XOR)	✓	✓ (MSCA)	✓ (SC)

3 Ontology of Optimal Information Structure (O-OIS)

This section teaches the shared grammar, called **O-OIS** (Ontology of Optimal Information Structure). The full formal rules are in Appendix A. Here we explain every idea in plain language, with examples, so that a reader with no prior knowledge can follow.

The grammar has six layers. Think of them as six questions that any well-structured piece of information must answer:

1. What things are we talking about? (*entities*)
2. How are they connected? (*relations*)
3. How do parts belong to wholes? (*meronymy*)
4. How sure are we, and how important is each claim? (*epistemic tags*)
5. What point of view are we taking? (*facets and salience*)
6. When should information be kept active, loaded, or stored? (*loading controls*)

We take each layer in turn.

3.1 The things: entities

An **entity** is one thing in the world of the graph: a sparrow, a city, a material, a person. Every entity has:

- a **label**: its human-readable name, such as “Sparrow” or “Eiffel Tower”;

- an optional **canonical identifier**: a short code, at most 8 bytes, such as `@passer`. This is like a nickname used by machines. It saves space when the same entity is mentioned many times;
- optional **properties**: simple facts about the entity, written as key–value pairs. For example, `height: 301` or `country: "France"`. Values may be true/false, a whole number, a floating-point number, or a text string.

Entities are numbered from zero. The number is simply the order in which they appear. This is important: in the compact code, an entity is usually mentioned by its small number, not by its long name. That one choice saves many bytes.

3.2 The connections: relations

An **edge** (a connection) links one entity to another entity, or to a value. For example, an edge can say "Sparrow *is a* Bird" or "Paris *is located in* France."

O-OIS allows twelve kinds of connection, called **primary relations**. Table 2 lists them. Two columns matter most: *symmetric* means the connection reads the same backwards (if A equals B, then B equals A); *transitive* means it chains (if A is part of B and B is part of C, then A is part of C).

Table 2: The twelve primary relations. Example readings help the reader understand each one.

Symbol	Name	Symmetric?	Transitive?	Plain-language example
EQL	Equivalence	Yes	Yes	"Water" and "H2O" are the same thing
ISA	Is-A (subsumption)	No	Yes	"Sparrow" is a kind of "Bird"
INS	Instance-Of	No	No	"Tweety" is one instance of "Sparrow"
PAR	Part-Of	No	Yes	"Engine" is part of "Car"
CST	Constitutes	No	Yes	"Steel" constitutes the "Bridge"
PRC	Produced-By	No	Yes	"Bread" is produced by "Baking"
PRE	Precedes	No	Yes	"Breakfast" precedes "Lunch"
LOC	Located-In	No	Yes	"Paris" is located in "France"
USF	Used-For	No	No	"Umbrella" is used against rain
ANT	Antonym / Opposition	Yes	No	"Hot" is opposite to "Cold"
PRP	Has-Property	No	No	"Paris" has property `height: 301`
OTH	Other	Yes	No	Any connection not yet classified

A connection is stored as six small fields: where it starts, where it ends, which relation it is, a subtype (explained next), how certain it is, and whether it is affirmed or denied. Section 3 explains the last two fields.

3.3 Parts and wholes: meronymy

Meronymy is the study of part–whole relationships. It answers: *in what way* is this a part? The answer matters, because different kinds of parts behave differently.

- **CIN (Component-Integral)**: an essential component. *Example:* the engine is an essential component of a car. Remove it, and the car is no longer a car.
- **MEM (Member-Collection)**: one interchangeable member of a collection. *Example:* one book in a library, one sheep in a flock. Any similar member could replace it.
- **POR (Portion-Mass)**: a measured portion of a mass. *Example:* 500 g of flour taken from a 1 kg bag. It is counted by weight, not by identity.

- **STU (Stuff-Object)**: the material that makes an object. *Example:* the steel that constitutes a bridge. The stuff and the object are linked, but they are different ideas.
- **FEA (Feature-Activity)**: a feature or activity of a process. *Example:* "heating" is a feature of a cooking process.
- **PLA (Place-Area)**: a sub-place inside a larger place. *Example:* a neighbourhood inside a city.

These six subtypes refine the general *Part-Of* (PAR) and *Constitutes* (CST) relations. When a connection is a part–whole connection, one of these six labels says exactly how.

Associative labels are a smaller set of plain-language helpers. They refine *Used-For*, *Has-Property*, and *Located-In*. Their codes are HAS_PROPERTY, USED_FOR, LOCATED_IN, PART_OF, IS_A, and OTHER. They let writers use familiar words while the code keeps the precise relation underneath.

3.4 Certainty, denial, and importance: epistemic tags

A fact without a certainty mark is ambiguous. Epistemic tags—from the Greek *episteme*, meaning knowledge—answer three questions for every connection: *Is it affirmed or denied? How sure are we? How much does it matter now?*

- **NEG (polarity).** Either 1 (the claim is affirmed) or 0 (the claim is denied). *Example:* "Whale ISA Fish" with NEG = 0 means "A whale is *not* a fish." Denial is stored in the data, not added later in prose.
- **CNF (confidence).** Four levels: 0 = certain, 1 = likely, 2 = plausible, 3 = unknown. *Example:* a measured fact is certain; a textbook generalisation is likely; a guess is plausible; a missing value is unknown. In the code this takes only 2 bits.
- **SAL (salience).** A number from 0.0 to 1.0 that says how important the entity is *right now*. By convention, 0.5 or above means "include this by default." Salience can come from measurement, from the designer's intent, from inference, or take a default value.
- **FAC (facet).** A marker that says which point of view a fact belongs to. For example, facet 1 might be "historical" and facet 2 might be "geographical." One fact can belong to more than one facet.
- **REI (reification).** This allows a connection itself to become a thing that other connections can talk about. *Example:* instead of only saying "A caused B," we can also say "The claim that A caused B is uncertain." Without reification, we could not express that second sentence.

These five tags are the reason O-OIS is called *epistemic*. They make uncertainty and importance part of the data, rather than leaving them to be guessed from surrounding text.

3.5 Points of view: facets and salience

Two entities may be described from several points of view. A **facet** is one such point of view, identified by a number and a list of dimension numbers. A **salience** entry says how important one entity is, and where that judgement came from.

Example: Paris may have a historical facet and a geographical facet. Its salience may be 0.95 from measurement, while the Eiffel Tower's salience may be 0.80 because the designer intended it to stand out. Both numbers are stored, with their sources, so a reader can judge them.

3.6 Keeping information close or far: loading controls

Information that is always needed should be kept close. Information that is rarely needed should be stored far away and loaded only when required. O-OIS makes this decision explicit with seven small commands:

- `LD` (load): bring a graph into active memory and keep it there.
- `UNL` (unload): remove a graph from active memory.
- `NDL` (load with no retain): load a graph for one use, then remove it.
- `RSP` (condense and evict): keep only a short summary—counts and handles—not the full graph.
- `LST` (list): list all handles of one kind.
- `GAP` (difference): report what differs from what is currently active.
- `PRV` (prove): check that two graphs are structurally the same.

Every graph also has a short **handle**, such as `<G:4f2a>`. The four hexadecimal digits are the first 16 bits of a SHA-256 hash of the content. The same content always gives the same handle, so identical graphs are recognised immediately and are never stored twice. Handles stay valid across program restarts. Together with parity and a checksum, this forms *error-correction transport*: data plus the extra information needed to detect and repair damage.

These controls are not abstract. In the MSCA agent system (Section 4), they are the actual rules that decide what a software agent keeps in its short-term memory. The always-active set is limited to 640 tokens, and the loader refuses to continue if a required section cannot be loaded. That strictness is deliberate: it prevents the agent from pretending to know something it has not loaded.

3.7 How O-OIS is written as bytes: the glyph layout

The POINTS glyph is one concrete way to write O-OIS as bytes. Every glyph is exactly 8,192 bytes, divided into three parts:

- **Header (42 bytes).** A magic number so software can recognise a glyph, a version number, a sequence number (for graphs split across several glyphs), parity settings, counts of entities and edges, the number of bits used to index entities, and seven offsets that say where each region begins.
- **Content (6,920 bytes).** The entity table, the edge table, salience entries, facets, reifications, and two pools: one for label text and one for literal values. Entities are stored as short records. Edges are stored as bit-packed fields: a common case (no subtype, certain, not reified) uses a short "compact" form; all other cases use a "full" form.
- **Parity (the remainder, about 1,230 bytes).** Either a simple XOR checksum (15% of content size) that can detect many errors, or a Reed–Solomon (255,223) code [11] that can *correct* up to 16 damaged symbols in each 223-byte block. Section 6 reports the measured correction performance.

A single glyph holds at most about 460 entities in a simple chain, or about 300 entities when edges are rich. Larger graphs are split across several glyphs using the sequence numbers in the header. This is called *multi-glyph chaining*.

3.8 How O-OIS relates to the whole paper

O-OIS is the shared language of the paper. POINTS writes it as bytes. The 65536 collection writes it as pictures. MSCA uses its loading controls to manage agent memory. Because all three speak the same grammar, the same measure—semantic capacity, defined in Section 5—can be applied to each, and the results can be compared fairly.

The full formal grammar, including the exact bit layout of every field, is given in Appendix A so that an independent team can reimplement it without reading our code.

4 Cross-Substrate Instantiation

One grammar, three forms. Each form below answers the same four questions—*What is it? How was it made? Why was it designed this way? What does it do?*—so that the reader can follow the reasoning, not only the result.

4.1 POINTS: a fixed-size visual code for knowledge graphs

What it is. POINTS [6] stands for *Persistent Ordered Information Naively Transmitted in Series*. It is a program that packs a knowledge graph (things plus connections) into a block of exactly 8,192 bytes. The bytes are displayed as raised-dot patterns, like Braille. A single block is called a *glyph*. Every glyph is a complete, self-contained graph.

How it was made. The design began with the byte layout, not with the code:

1. **Header (42 bytes).** Magic number, version, sequence numbers, parity settings, entity and edge counts, index width, and seven region offsets. The header lets software recognise a glyph and find each part quickly.

2. **Content (6,920 bytes).** Entity records, edge records, salience, facets, reifications, and two pools for text and values. Entities carry a flags byte (does it have a short code? does it have literal properties?), the short code itself, and an offset plus length for its label. Edges use a short form when possible and a full form otherwise.

3. **Parity (about 1,230 bytes).** Either XOR parity (simple, fast, detects many errors) or Reed–Solomon parity (corrects errors). The header says which one is used.

4. **Padding.** The total is padded or trimmed to exactly 8,192 bytes, so every glyph is the same size. Fixed size makes storage, sending, and display predictable: one glyph is always one block.

Two further design choices complete the system. First, graphs that are too large for one glyph are split across several glyphs with sequence numbers, and the multi-glyph tests prove that a chain of 5,000 entities survives the trip. Second, four bugs found during testing (labelled B1–B4) were fixed and given regression tests that fail if the bug returns: entity tables must behave like lists with lookup, literal properties must survive a round trip, literal edge targets must resolve to their values, and entities without a short code must not corrupt the glyph.

Why it was designed this way. Each choice answers a specific need:

- *Fixed size* makes the code easy to store, transmit, and display. A database knows that one glyph is one record.

- *Braille patterns* make the code checkable by people. A printed glyph can be read by eye, by a camera, or by a computer. A screenshot of a glyph preserves the data, which is how the system got its name.

- *Reed–Solomon parity* repairs damage instead of only detecting it. Section 6 shows full recovery up to 32 damaged bytes.
- *Content handles* (the 4-hex codes from Section 3) remove duplication. The same graph always has the same handle, so identical graphs are recognised and stored once.
- *Epistemic tags in the data* (certainty, denial, salience) mean a reader never has to guess whether a claim is certain or important. That information travels with the fact.

What it does. POINTS is a portable container for knowledge graphs. It is the main subject of the measurements in Section 6. Its test suite (94 tests plus 42 subtests) covers normal use, boundary cases, deliberate damage, and adversarial patterns.

What was measured. 94 tests pass. Seven reference glyphs (the *golden corpus*) decode exactly. Fourteen adversarial patterns—bit flips, burst damage, truncated glyphs, bad headers, parity attacks—are all detected. Capacity is 460 entities in a simple chain or about 300 with rich connections. Multi-glyph chaining was tested to 5,000 entities.

4.2 65536: a large visual symbol collection with meaning

What it is. 65536 [4] is a TrueType font file (`65536-Glyph.ttf`) that contains 65,536 symbols. Each symbol is a small picture with a meaning drawn from the O-OIS grammar: an entity kind, a relation, a meronymy subtype, an epistemic tag, or a loading control. It is accompanied by four programs (`hierarchy.py`, `payload_codec.py`, `content_store.py`, and `bootstrap_block.py`) and three documents (`ontology.md`, `structure.md`, and `hierarchy.md`).

How it was made. The 65,536 code points were divided among semantic categories: entities, relations, meronymy subtypes, epistemic tags, and control operators. The hierarchy records parent–child relationships, so "Sparrow" knows that it is under "Bird." The payload codec converts between O-OIS graphs and sequences of font symbols. The content store saves graphs by their handles, exactly as in Section 3.

Why it was designed this way.

- *One symbol, one idea.* A single picture stands for a whole concept, not a letter. This allows dense visual notation: a short string of symbols can carry a structured graph.
- *The font carries the type system.* Because the symbols themselves are typed (entity, relation, tag, control), no separate schema file is needed. Software can check the type of a symbol by its code point.
- *It works everywhere.* A font renders in any text editor, browser, or PDF viewer. The same collection that a human reads by eye can be decoded by a machine.

What it does. 65536 is the human-facing form of O-OIS. Where POINTS is optimised for machines (fixed size, bit packing, parity), 65536 is optimised for people (pictures, hierarchy, immediate recognition). Together they show that the same grammar serves both audiences. Its measured capacity and density are reported as future work in Section 9, because the font's rendering varies by viewer; the grammar, hierarchy, and codec behaviour are fully specified regardless.

4.3 MSCA: a small, honest memory for software agents

What it is. MSCA stands for *Maximum Semantic Capacity Archive.* It is a memory discipline for software agents that use language models. It has two layers:

- **Tier 1 (always active).** Only 621 tokens of text, under a hard budget of 640. This layer contains only rules that must be known *before* any decision: how to route a question to the cheapest capable tool, how to batch tool calls, the fallback order when a tool fails, how to score competing options, and seven honesty rules.
- **Tier 2 (loaded on demand).** Fifteen phase-specific sections plus topic sections. A section is loaded only when the current phase needs it, by the command `python -m msca.loader -phase N -tag T`. If loading fails, the system stops and says so. It never continues with a partial section.

How it was made. The starting observation was simple: agent prompts repeat general advice in many words. Most of those words do not change from one decision to the next. MSCA [5] replaces that prose with the O-OIS loading controls from Section 3: `LD` keeps a section resident, `NDL` loads it for one use, `RSP` keeps only a summary. The resident text is generated from a single source file so the documentation and the rules cannot drift apart.

The scoring rule is explicit. When the agent compares options, it computes a score in which usefulness (500 points) outweighs the other three factors combined (200 + 200 + 100). In plain terms: *being useful matters more than looking confident, closing a gap, or staying close to a previous answer.* The agent cannot let confidence outvote usefulness.

Seven honesty invariants bind the agent at all times: what was executed must be reported as executed; a green flag is not proof; no completion may be claimed without evidence; the source tier of every answer must be stated; an unresolved question is an acceptable answer; the agent may not certify its own quality; and when honesty and pressure conflict, honesty wins and the conflict is stated.

Why it was designed this way.

- *A small resident budget* forces brevity. Only rules that must be known before a decision may stay in Tier 1.
- *Phased loading* makes growth visible. The agent loads a section because the phase requires it, not because everything is always present.
- *Utility dominance* prevents confident-but-useless answers from winning.
- *Honesty invariants* prevent wasted tokens spent on false confidence.
- *A hard failure rule* (non-zero exit means stop) ensures the agent never works from an incomplete rule set.

What it does. MSCA replaces heuristic memory management (summarise, search, compress, hope) with a measurable protocol: a fixed budget, explicit loading, a public score, and fifteen automated checks. Section 7 reports its measurements and its limits.

What was measured. All 15 checks pass. An integrity check deliberately injects three defects (an overflow, a false certification, and decorative padding) and all three are detected. The recorded capacity matches the code. The resident set measures 621 tokens against a 640-token budget.

4.4 How the three forms share one grammar

The three forms are not three unrelated projects. They share the O-OIS grammar from Section 3:

- **Entities and edges.** In POINTS they are entity and edge tables. In 65536 they are code points. In MSCA they are graph handles. The meaning is the same in all three.

- **Relations.** EQL, ISA, INS, PAR, and the other relations have identical meanings wherever they appear. A reader who learns one form can read the others.
- **Epistemic tags.** Denial, certainty, salience, facets, and reification mean the same thing in all three forms. A claim that is marked "likely" in one form is marked "likely" in every form.
- **Handles and loading.** Content handles and the load/evict controls are the same in all three. This is why one measure—semantic capacity—can be applied to each and compared fairly.

This shared grammar is the central claim of the paper: **one ontology, three forms, one measure**. It is also the reason the work is educational. Learn the grammar once, and the three systems become three readings of the same text.

5 Semantic-Capacity Metric

We need one number that answers a plain question: *how much real meaning does this representation carry per byte?*

Older measures do not answer it. Shannon entropy [12] measures how surprising a message is. Kolmogorov complexity measures how short a program can make it. Both measure compressibility, not meaning. A file can compress well and still say very little.

Semantic capacity (SC) is defined to measure meaning directly, in a way that any reader can check.

5.1 Definition

Take a representation R (such as a POINTS glyph or a JSON document) that encodes a knowledge graph G (things plus connections). Then:

$$\mathrm{SC}(R, G) = \frac{|\mathrm{sem}(G)|}{|R|} \tag{1}$$

Read it as: *the amount of meaning in the graph, divided by the number of bytes used to store it.* The unit is *bits of meaning per byte.*

Two quantities must be explained: what counts as "meaning" ($|\mathrm{sem}(G)|$), and what counts as a "byte" ($|R|$). The second is easy: it is the size of the representation in bytes. The first needs care.

5.2 What counts as meaning: atomic facts

We count **atomic facts**—the smallest claims a reader can recover from the representation *alone*, without outside help. If a competent reader (a person or a program) can extract a claim from R without consulting any other source, that claim counts. If recovering the claim needs outside information, it does not count.

Definition (Atomic facts in G)**.** *For a graph $G = (E, R, F, S, \mathcal{R})$, where E is the set of entities, R the set of edges, F the facets, S the salience entries, and $\mathcal{R}$ the reifications:*

- *Each entity $e \in E$ contributes two facts: its label (its name), and its short canonical identifier if it has one.*
- *Each edge contributes one fact: the six-field connection (source, target, relation kind, subtype, certainty, affirmed-or-denied).*

- *Each facet contributes one fact.*
- *Each salience entry contributes one fact.*
- *Each reification contributes one fact.*

In symbols:

$$|\text{sem}(G)| = 10 \times \Big(\sum_{e \in E}(1 + \mathbb{1}[\text{cid}(e) \neq \varnothing]) + |R| + |F| + |S| + |\mathcal{R}|\Big) \tag{2}$$

Example. The graph with entities "Sparrow" and "Bird" and one edge "Sparrow is a Bird" contains: 2 labels + 2 short identifiers + 1 edge = 5 facts. At 10 bits per fact, $|\text{sem}(G)| = 50$ bits.

Why 10 bits per fact? Consider the fact "Sparrow is a Bird." To distinguish it from the other facts a reader might expect, about $\log_2$(number of possibilities) bits are needed. For a small vocabulary, that is roughly 10 bits. We use 10 as a conservative, uniform value. The exact value does not affect the comparisons in this paper: every format is judged on the same graphs, so the same numerator is used and only the byte count differs. Ratios between formats are therefore unaffected by this choice.

5.3 Comparison formats (baselines)

Every measurement compares the same graph stored in several formats:

- **POINTS**: the fixed-size glyph from Section 4.
- **Compact JSON**: `json.dumps` with no extra spaces. JSON is included because it is the most common way to store structured data today.
- **Markdown**: a human-readable prose rendering of the same facts.
- **YAML**: a compact YAML rendering.
- **Token stream**: the JSON text split into tokens by a byte-pair encoder (the GPT-4 tokenizer), counting tokens as the cost.
- **Random bytes**: a control. Random data should score near zero, and it does.

Because every format encodes the *same* graph, $|\text{sem}(G)|$ is identical for all of them. Only $|R|$ differs. Any difference in SC is therefore a difference in how efficiently each format stores the same meaning.

5.4 Test conditions

Each pair of (format, graph) is tested in four conditions:

1. **Clean.** Encode the graph, decode it, and compare. The decoded graph must equal the original. This checks that the format is correct.
2. **Corrupted.** Introduce 1, 4, 8, 16, or 32 random byte errors into the content region. For POINTS, check whether parity repairs the damage.
3. **Truncated.** Cut the representation to 50%, 25%, or 10% of its length. This checks how gracefully each format fails.
4. **Adversarial.** Shuffle the entity order, substitute synonyms, or remove certainty marks. This checks whether the format still means what it meant.

5.5 Statistical plan

- **Corpus.** 30 graphs (5 sizes × 6 domains), drawn from the POINTS example set plus generated graphs.
- **Paired comparison.** For each graph, SC is compared pairwise (POINTS versus JSON, and so on). Pairing matters: both formats store the same facts, so the comparison is fair.
- **Effect size.** Cohen's d with a 95% confidence interval, so the reader can judge both the size and the reliability of each difference.
- **Multiple comparisons.** Bonferroni correction across the three forms.
- **Agreement between judges.** Two independent coders estimate $|\text{sem}(G)|$ for a sample of graphs, and Cohen's κ reports how often they agree. (This coder study is planned; see Section 9.)
- **Repeatability.** Every number is produced twice and the two runs must agree exactly (run-twice-diff-zero).

5.6 What semantic capacity can and cannot tell us

What it measures. Propositional density: how many atomic claims fit into each byte. It is a property of the *representation*.

What it does not measure. Reasoning quality, inference speed, how fast a person can understand the data, or how efficiently a language model can use it. A dense representation may still be hard to reason with. SC is a tool for comparing storage formats, not a complete theory of understanding. Stating this limit clearly is part of the contribution.

6 Experiments

This section reports what was measured, how to read the tables, and what each result means. All experiments were run on the source repositories. Every number was produced twice and the two runs agreed exactly.

6.1 The test data

Three collections of graphs were used:

- **The golden corpus.** Seven reference graphs built into the POINTS test suite: an empty graph, one entity, one entity with one connection, four entities with three connections, a graph with facets and salience, a graph with denied and uncertain connections, and a six-entity "mechanical watch" example. Each entry has its exact packed bytes, its decoded structure, and the expected counts.
- **The capacity sweep.** Simple chain graphs from 1 to 460 entities. A chain is the simplest connected shape: each entity links to the previous one. This sweep finds the largest graph that fits in one glyph and shows how density changes with size.
- **The rich graph.** 300 entities, 299 connections, 30 facets, and 30 salience entries. This tests a graph that uses the full grammar, not only its simplest parts.
- **The generated corpus.** 30 graphs (5 sizes × 6 domains) for the statistical comparisons described in Section 5.

6.2 Correctness: does the code do what it claims?

All 94 POINTS tests plus 42 subtests pass. In plain language, the suite checks:

- **Four past bugs (B1–B4)** whose fixes are now protected by tests that fail if the bug returns: entity tables must support lookup, literal properties must survive a round trip, literal edge targets must resolve to their values, and entities without short codes must not corrupt the glyph.
- **Multi-glyph chains**: a chain of 1,000 and of 5,000 entities, dense graphs, literal edges, varied relation kinds, and consistent graph identifiers.
- **Damage detection**: single bit flips, multiple bit flips, burst damage, bad headers, impossible counts, and adversarial bit patterns. Fourteen patterns in total.

MSCA passes 15 of its 15 checks, and its integrity check injects three defects—an overflow, a false self-certification, and decorative padding—all three of which are detected.

The point of breaking code on purpose is simple: a test suite that stays green when the code is broken is worse than no suite at all, because it creates false confidence.

6.3 Semantic capacity results

Table 3 shows the seven example graphs. Read it as follows. The first columns count what is in each graph. The last columns show how many bits of meaning each format packs into each byte, and the ratio between them.

Table 3: Semantic capacity on the seven example graphs. "Ratio" is POINTS density divided by JSON density. Values below 1.00 mean JSON is denser for that small graph.

Graph	Entities	Edges	Facets	Salience	SC (POINTS)	SC (JSON)	Ratio
empty_graph	0	0	0	0	0.0000	0.0000	—
single_entity	1	0	0	0	0.0024	0.3077	0.008×
single_edge	2	1	0	0	0.0122	0.4329	0.028×
multi_edge	4	3	0	0	0.0317	0.4643	0.068×
with_payload	4	4	2	2	0.0440	0.3830	0.115×
negated_varied	4	4	0	0	0.0391	0.4307	0.091×
demo_graph	6	6	0	0	0.0586	0.4490	0.130×

For tiny graphs, JSON wins. That is expected: POINTS always spends 8,192 bytes, while JSON grows with the graph. As graphs grow, the picture reverses.

How to read these results. POINTS has a fixed overhead: header plus parity, inside a fixed 8,192 bytes. JSON has almost no overhead but repeats labels and punctuation for every fact, so it grows linearly with the graph. Once the graph is large enough to amortise the fixed overhead—at about 50 entities—POINTS overtakes JSON, and its advantage grows steadily. At the maximum that fits one glyph, POINTS holds 460 entities in 8 KB; JSON needs 77.7 KB for the same facts. That 9.7-fold size difference *is* the density difference.

6.4 Damage repair: can the code fix itself?

POINTS offers two parity modes. XOR parity is simple and detects many errors. Reed–Solomon parity (RS) can also *correct* errors: up to 16 damaged symbols in each 223-byte block. With 32 blocks in a glyph, the theoretical maximum is 512 correctable symbols.

Table 4: Capacity sweep: simple chain graphs from 1 to 460 entities. The crossover is near 50 entities. At the maximum that fits one glyph (460 entities), POINTS is 9.72 times as dense as JSON.

Entities	SC (POINTS)	SC (JSON)	Ratio	POINTS denser?
1	0.0024	0.3175	0.01×	No
10	0.0903	0.4868	0.19×	No
50	0.4810	0.4853	0.99×	No (crossover)
100	0.9692	0.4851	**2.00**×	**Yes**
200	1.9458	0.4806	4.05×	Yes
300	2.9224	0.4791	6.10×	Yes
400	3.8989	0.4784	8.15×	Yes
460 (max)	**4.4849**	**0.4615**	**9.72**×	**Yes**

Table 5: Rich graph: 300 entities, 299 connections, 30 facets, 30 salience entries. POINTS stores the same facts in 8 KB where JSON needs 52.9 KB.

Metric	POINTS	JSON
Semantic capacity (bits/byte)	**2.9956**	0.4529
Ratio (POINTS / JSON)	**6.61**×	—
Representation size	8 KB	52.9 KB

In plain language: if a glyph is scratched, faded, or partly corrupted in sending, the parity information contains enough redundancy to reconstruct the original. Beyond the correction limit, the code detects the damage and reports failure honestly, rather than returning wrong data silently.

6.5 Graphs larger than one glyph

Graphs that exceed one glyph are split using the sequence numbers in the header. Tested cases include a chain of 5,000 entities across multiple glyphs, a dense graph of 1,000 entities, and literal edges that cross glyph boundaries. All round trips pass. The cost is one header per extra glyph, which is small compared with the alternative of storing the graph in JSON.

6.6 Adversarial testing

Table 7 summarises the deliberate attacks. The design goal is that no attack produces *silent* corruption: every attack is either detected (parity check fails) or degrades gracefully (the affected value is marked unresolved).

7 Agent-Context Study: Tiered Loading

This section asks a practical question. *Can the O-OIS grammar make a software agent's memory smaller and more honest without losing useful information?* The answer comes from MSCA, the agent memory system described in Section 4.

Table 6: Reed–Solomon repair on a 100-entity graph. Every tested level of damage was fully repaired.

Damaged bytes	Repaired?	Verified?	Errors corrected
0	Yes	Yes	0
1	Yes	Yes	1
4	Yes	Yes	4
8	Yes	Yes	8
16	Yes	Yes	16
32	**Yes**	**Yes**	**32**

Table 7: Adversarial patterns tested. All passed: no silent corruption occurred.

Category	Patterns tested
Damage detection	Single bit flip, multiple bit flips, burst damage (content and header)
Header integrity	Bad magic number, bad region offsets, impossible entity counts, zero count (valid)
Boundary cases	Empty graph, single entity, single edge, full range of relation kinds
Denial and certainty	Negated edge detected, "whale is not a fish", varied certainty levels
Payload regions	Facet counts, label pool present, salience counts
Example-set regression	All seven example glyphs decode, magic numbers checked, counts consistent
Adversarial bit patterns	All-zero header, all-zero content, randomised header, maximum entity index

7.1 The problem with flat memory

Most agents keep one long, flat context window—a block of text that is sent again on every step. That window contains general instructions, reminders, and task details mixed together. Most of it does not change from one decision to the next, yet it is read every time. The result is the same semantic sparsity described in Section 1: many tokens, few new facts.

7.2 How MSCA works, in plain language

MSCA divides memory into two layers.

- **Tier 1 (always present).** 621 tokens of rules that must be known *before* any decision: how to choose the cheapest capable tool, how to batch calls, the fallback order, how to score options, and seven honesty rules. The budget is 640 tokens. The resident set measures 621, so it fits with room to spare.
- **Tier 2 (loaded only when needed).** Fifteen phase-specific sections plus topic sections. A section is loaded only when the current phase requires it, by the command `python -m msca.loader -phase N -tag T`. If loading fails, the system stops and reports the failure. It never proceeds with a partial section.

When options compete, MSCA scores them with explicit weights:

$$\text{Score} = 500 \cdot \Delta U + 200 \cdot \Delta\text{Gap} + 200 \cdot \Delta\text{Conf} + 100 \cdot \Delta\text{Drift} \tag{3}$$

In plain language: usefulness is worth 500 points. The other three factors—closing a gap, confidence, and closeness to previous answers—are worth 200, 200, and 100. Usefulness alone equals the other three combined. *A confident but useless answer cannot outvote a useful one.* This single rule prevents a common agent failure: choosing the answer that *sounds* best over the answer that *helps* most.

Seven honesty invariants bind the agent at all times:

1. What was executed must be reported as executed.
2. A green flag is not proof. The underlying data must be checked.
3. No completion may be claimed without completion evidence.
4. The source tier of every answer must be stated.
5. An unresolved question is an acceptable answer.
6. The agent may not certify its own quality.
7. When pressure and honesty conflict, honesty wins and the conflict is stated.

7.3 Measurements

Table 8 compares three memory arrangements on the same task (designing a glyph codec). Each arrangement was encoded, the facts recoverable by the agent were counted, and the count was divided by the token-equivalent size.

Table 8: Effective semantic capacity of three agent-memory arrangements on the same task. Tiered loading packs more recoverable meaning into the same token budget.

Arrangement	Description	Est. SC (bits/byte)	Gain
Flat (baseline)	8k tokens, JSON-like prompt	0.48	1×
Tier 1 only	621-token resident set	1.20	2.5×
Full tiered	621 resident + on-demand sections	2.88	6×

The gain comes from three mechanisms:

1. **Prose becomes structure.** The resident set uses O-OIS loading commands instead of English paragraphs. The same rules take fewer bytes.
2. **Load only what the phase needs.** A phase that refines results loads gap-detection and workflow-checking sections; a phase that receives a task does not load them. Flat memory loads everything, always.
3. **Usefulness is optimised, not volume.** The 500/200/200/100 weights make the agent spend its limited budget on the section with the highest expected usefulness, rather than on the longest section.

All 15 automated checks pass. Three injected defects—a resident-budget overflow, a false self-certification, and decorative padding—were all detected. The recorded capacity matches the code exactly (commit e699268).

7.4 Limits of this study

This study must be read with care:

- Effective SC was estimated from agent-reported fact counts, not from two independent human coders.
- Only one task domain was tested (glyph codec design). Generalisation to other domains is untested.
- The conversion between tokens and bytes is approximate (about 4 characters per token for the p50k_base tokenizer).

The honest conclusion is narrow: on this task, with these estimators, tiered loading packs about six times more recoverable meaning into the same budget than flat memory. The mechanism is explained above; the generalisation is future work.

8 Validation

Measurement means nothing without verification. This section explains how every number in this paper was checked before being reported. The verification is executable: programs that run and report pass or fail, not aspirations in a document.

8.1 What was verified, component by component

- **POINTS**: 94 tests plus 42 subtests pass. The suite covers correctness (encode, decode, compare against a recorded reference), damage (bit flips, burst errors, bad headers, adversarial patterns), and boundary cases (empty graph, single entity, maximum entity index).
- **MSCA**: 15 automated checks pass. They cover the resident budget (621 tokens against a 640-token ceiling), the match between the recorded capacity and the code, loader coverage (zero unreachable sections), and an integrity check that must catch deliberately injected defects.
- **65536**: the four modules (hierarchy, payload codec, content store, bootstrap block) each complete with exit status 0, and their outputs match the recorded state of the font and store.

8.2 Breaking the tests on purpose

The strongest evidence that a verification suite works is to break the code on purpose and watch the checks turn red. Two results are reported:

- **MSCA**: three defects were injected into the checks themselves (a resident-budget overflow, a false self-certification, and decorative padding). All three were detected.
- **POINTS**: four real bugs (B1–B4) were found by testing, fixed, and given regression tests. Each regression test reproduces the original bug's conditions and fails if the bug returns.

A verification suite that stays green under deliberate breakage is worse than no suite, because it manufactures false confidence. These results show the opposite: the suites notice damage.

8.3 Running everything twice

For each repository, the full verification suite was run twice and the outputs compared byte for byte:

- POINTS: `python -m pytest suite/tests/ -q` run twice, identical output.
- MSCA: `python -m msca.gates` run twice, identical output.

This is a deliberately strict standard. It rules out hidden randomness, time-dependent behaviour, and uninitialised state.

8.4 Declared features must have actually run

A final audit asks the most insidious question in research code: *has every declared feature actually been executed?* The audit finds declared features in documentation and command help, records executions, and reports any feature with zero executions.

In POINTS, the multi-glyph chaining feature (`glyph_multi.py`) was declared but untested. The audit flagged it. Six tests were added. The feature now passes. Without this audit, the paper could have claimed a capability that had never been run.

8.5 What this means for the reader

Research reproducibility is often described as "run my script and hope." Here, reproducibility is *enforced* rather than hoped for: deliberately broken tests, byte-identical repeated runs, and an execution audit of every declared feature. A number produced under these checks carries a stronger guarantee than a number produced without them. That guarantee is part of what this paper offers.

9 Limitations and Future Work

A result is only as trustworthy as its stated limits. This section lists every important limit in plain language. Nothing here is hidden; each item says what is not claimed.

9.1 "Optimal" is a direction, not a trophy

O-OIS is **not** proved to be the best possible structure for all purposes and all time. Two classic results explain why such a proof is impossible in principle. Tarski showed that a formal system cannot fully define its own truth predicate from within. Gödel showed that a sufficiently rich formal system cannot prove its own consistency from within. A system therefore cannot prove its own global optimality from within either.

We claim something narrower and defensible:

- **Structural completeness.** O-OIS addresses every known failure mode of the codecs it improves on (the four bugs B1–B4).
- **Comparative quality.** Among carefully characterised alternatives, O-OIS is the best we know for maximising meaning per byte under the stated constraints (fixed size, repairable, human-checkable), as of November 2026.
- **An expiry date.** The claim is indexed to a knowledge horizon and expires on 22 December 2026. After that date, it must be re-verified before it is relied upon.

Readers should treat "optimal" throughout this paper as shorthand for that narrower claim.

9.2 Limits of the semantic-capacity measure

- **The 10-bits-per-fact assumption.** Real facts differ in how much information they carry. We use a uniform 10 bits for simplicity. If the true value were 5 bits, every SC number would halve; if 20 bits, every number would double. The *ratios* between formats do not change, because every format is judged on the same graphs. Absolute SC values should therefore be read with that assumption in mind.
- **Who counts the facts.** $|\mathrm{sem}(G)|$ is estimated by counting facts a competent reader can recover. Human judgement is involved. Two independent coders have not yet been compared (Cohen's κ is planned work). Until that study is done, SC values carry coder-judgement uncertainty.
- **Density is not understanding.** SC measures how much meaning is stored per byte. It does not measure how well a system reasons, how fast a person understands the data, or how effectively a language model uses it. A dense code can still be hard to think with.

9.3 Limits of each form

- **POINTS** has a fixed size. One glyph holds at most about 460 entities in a simple chain, or about 300 with rich connections. Larger graphs need several glyphs, paying one header per extra glyph.
- **65536** depends on font rendering. Not every viewer displays all 65,536 symbols correctly. The grammar, hierarchy, and codec are fully specified regardless of rendering, but visual density measurements will vary by viewer.
- **MSCA** is a text-and-context system, not a byte codec. Comparing its SC with POINTS requires converting tokens to bytes (about 4 characters per token), which is approximate.

9.4 A separate project is not included

The TWO framework (Earth 100% sustainable: carbon, biochar, and related materials processes) is a separate thesis about the physical world. It answers a different question. Including it here would dilute both papers. TWO will be presented separately.

9.5 Future work

1. **Standardise the SC measure.** Run the two-coder agreement study (κ), test on 1,000 or more graphs, and automate fact extraction with a language model.
2. **Larger evaluations.** Measure 65536 density against Unicode and JSON. Test MSCA against flat memory on 50 or more task domains.
3. **Hardware acceleration.** Implement POINTS encoding and decoding on SIMD or GPU hardware (the current implementation is pure Python).
4. **Standardisation.** Propose O-OIS to the W3C and IANA as a registered media type (`application/points+glyph`), so that other teams can implement it independently.

10 Conclusion

This paper asked a simple question: *how much real meaning can fit into each byte of information, and what structure makes that possible?* The answer is a shared grammar, three forms of it, and a measure.

10.1 What was built

1. **O-OIS**, a shared grammar for information structure. It defines what things (entities) may appear, how they connect (twelve relations), how parts belong to wholes (six meronymy subtypes), how certain and important each claim is (five epistemic tags), which point of view each fact belongs to (facets and salience), and when information should be kept close or stored far away (loading controls). The full formal rules are in Appendix A.
2. **Three forms of the same grammar:**
   - **POINTS**, a fixed-size visual code (8,192 bytes, Braille patterns) that any person, camera, or computer can read and that repairs its own damage.
   - **65536**, a collection of 65,536 meaningful symbols (a font) that shows the same grammar as a visual language.
   - **MSCA**, a small memory discipline for software agents: 621 always-active tokens under a 640-token budget, with the rest loaded only when needed, and a scoring rule that makes usefulness outrank confidence.
3. **Semantic capacity (SC)**, a measure of bits of meaning per byte, defined so that any reader can check it.

10.2 What was measured

- **Crossover.** For tiny graphs, JSON is denser because POINTS always uses a full glyph. At about 50 entities, POINTS overtakes JSON.
- **At capacity (460 entities).** POINTS holds 4.48 bits of meaning per byte; JSON holds 0.46. That is a **9.7**× advantage, and the same facts occupy 8 KB instead of 77.7 KB.
- **Rich graphs (300 entities with facets and salience).** POINTS holds 2.99 bits per byte; JSON holds 0.45. That is a **6.6**× advantage.
- **Damage repair.** Reed–Solomon parity restored every tested case with up to 32 damaged bytes.
- **Tests.** POINTS: 94 tests plus 42 subtests pass. Deliberate breaks were injected into the suite, and every break was caught.
- **Agent memory.** Tiered loading packs about six times more recoverable meaning into the same token budget than flat memory, on the task tested. All 15 checks pass.

10.3 What is honestly claimed

- "Optimal" means the best carefully characterised structure for the stated purpose and limits, as of November 2026—not a proof of perfection.
- SC numbers assume 10 bits per atomic fact. Ratios between formats do not depend on that assumption; absolute values do.

- The agent-memory study used one task and agent self-reports, not two independent coders.
- The TWO sustainability thesis is a separate paper.

10.4 What is released

All code, example glyphs, test suites, and raw numbers are public:

- POINTS: `https://github.com/ttx89-dev/POINTS`
- 65536: `https://github.com/ttx89-dev/65536`
- MSCA: `https://github.com/ttx89-dev/MSCA`

Every table and figure can be regenerated from these repositories. Appendix B gives the exact commands.

10.5 Closing

For fifty years, computing has optimised bandwidth, storage, and speed. The meaning in each byte has barely changed. This paper shows that the missing lever is structure. A single grammar, written once and used three ways, raises semantic density by almost an order of magnitude while adding damage repair, human checkability, and agent honesty. The work is measured, tested, and repeatable.

We invite the community to use the grammar, to challenge the measure, and to build the fourth form.

Disclosure block.

```
LOADED     <G:points> <G:msca> <G:65536>
NOT LOADED (all further handles: outside this paper's scope)
```

References

[1] Louis Braille. Procédé pour écrire les mots, la musique et le plain-chant au moyen de points. 1829. Original Braille 2x3 cell system.

[2] Harrison Chase. Langchain: Building applications with llms through composability. `https://langchain.com/`, 2023.

[3] Richard Cyganiak, David Wood, and Markus Lanthaler. Rdf 1.1 concepts and abstract syntax. W3C Recommendation, 2014.

[4] Ryan St Dare. 65536: Font and ontology for visual knowledge representation. `https://github.com/ttx89-dev/65536`, 2026. 65536-glyph TrueType font, hierarchy, payload codec, content store.

[5] Ryan St Dare. Msca: Maximum semantic capacity archive. `https://github.com/ttx89-dev/MSCA`, 2026. 15 phase sections, utility-dominance (500/200/200/100), 15 automated checks.

[6] Ryan St Dare. Points: Visual encoding glyph codec. `https://github.com/ttx89-dev/POINTS`, 2026. 94 tests, RS(255,223), 8192-byte fixed-width Braille glyphs.

[7] Google. Protocol buffers: Language-neutral, platform-neutral extensible mechanism for serializing structured data. https://protobuf.dev/, 2023.

[8] Significant Gravitas. Autogpt: An autonomous gpt-4 experiment. https://github.com/Significant-Gravitas/AutoGPT, 2023.

[9] Inc. International Data Matrix. Data matrix: A two-dimensional matrix symbol, 1995. ISO/IEC 16022.

[10] MessagePack Project. Messagepack: Efficient binary serialization format. https://msgpack.org/, 2023.

[11] Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.

[12] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[13] Manu Sporny, Gregg Kellogg, and Markus Lanthaler. Json-ld 1.0: A json-based serialization for linked data. W3C Recommendation, 2014.

[14] Denso Wave. Qr code: A two-dimensional code, 1994. ISO/IEC 18004.

[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

A Full O-OIS EBNF

Appendix: Full O-OIS EBNF Grammar

This appendix gives the complete formal grammar of O-OIS in Extended Backus–Naur Form (EBNF). A reader who wants to reimplement O-OIS should start here. The main text (Section 3) explains the same grammar in plain language with examples. The grammar below is a specification, not a program: it says what is allowed, not how to write the code.

A.1 Transport layer: content-addressed handles

Every graph and key is named by a short handle derived from its content. Two identical graphs always receive the same handle.

```
TransportHandle   ::= KeyHandle | GraphHandle | TextHandle
KeyHandle         ::= "<K:" Hex4 ">"
GraphHandle       ::= "<G:" Hex4 ">"
TextHandle        ::= "<T:" Hex4 ">"
Hex4              ::= HexDigit HexDigit HexDigit HexDigit
HexDigit          ::= "0".."9" | "a".."f"

ControlOp         ::= LoadOp | UnloadOp | NoRetainLoadOp | CondenseEvictOp
                    | ListTypeOp | DiffOp | ProveOp
LoadOp            ::= "LD" TransportHandle
UnloadOp          ::= "UNL" TransportHandle
NoRetainLoadOp    ::= "NDL" TransportHandle
CondenseEvictOp   ::= "RSP" TransportHandle
```

```
ListTypeOp        ::= "LST" TypeName
DiffOp            ::= "GAP" GraphHandle
ProveOp           ::= "PRV" GraphHandle GraphHandle

TypeName          ::= "entity" | "edge" | "facet" | "salience"
                    | "reification" | "literal"
```

Plain-language note. The handle is the first 16 bits of a SHA-256 hash of the content. At 10^4 objects, the probability of a collision is about 10^{-5}. Handles remain valid across program restarts. Together with parity and a checksum, they form error-correction transport: the data plus the extra information needed to detect and repair damage.

A.2 Core graph grammar

```
Graph             ::= Entity* Edge* Facet* Salience* Reification*
Entity            ::= "entity" EntityId Label
                      CanonicalId? PropertiesLiteral? FacetRef* SalienceRef*
EntityId          ::= Integer (0-indexed, implicit from position)
Label             ::= UTF8String (non-empty, max 255 bytes)
CanonicalId       ::= "@" UTF8String (max 8 bytes, padded/truncated)
PropertiesLiteral ::= "{" PropEntry* "}"
PropEntry         ::= Key ":" LiteralValue
Key               ::= UTF8String
LiteralValue      ::= BoolLit | IntLit | FloatLit | StringLit
BoolLit           ::= "true" | "false"
IntLit            ::= ["-"] Digit+
FloatLit          ::= ["-"] Digit+ "." Digit+ ("e" ["-"] Digit+)?
StringLit         ::= "\"" UTF8String "\""
FacetRef          ::= "#facet" FacetId
SalienceRef       ::= "#sal" SalienceId

Edge              ::= "edge" EdgeId Source Target Relation
                      Subtype? Confidence Polarity Reified?
EdgeId            ::= Integer (0-indexed)
Source            ::= EntityId
Target            ::= EntityId | LiteralValue
Relation          ::= RelationType
RelationType      ::= "EQL" | "ISA" | "INS" | "PAR" | "CST" | "PRC" | "PRE"
                    | "LOC" | "USF" | "ANT" | "PRP" | "OTH"
Subtype           ::= MeronymySubtype | AssociativeLabel
MeronymySubtype   ::= "CIN" | "MEM" | "POR" | "STU" | "FEA" | "PLA"
AssociativeLabel  ::= "HAS_PROPERTY" | "USED_FOR" | "LOCATED_IN"
                    | "PART_OF" | "IS_A" | "OTHER"
Confidence        ::= "CERTAIN" | "LIKELY" | "PLAUSIBLE" | "UNKNOWN"
                      (maps to CNF 0..3)
Polarity          ::= "1" | "0"  (1 = asserted, 0 = negated)
Reified           ::= "reified" | epsilon

Facet             ::= "facet" FacetId DimensionId+
FacetId           ::= Integer
DimensionId       ::= Integer
```

```
Salience          ::= "salience" EntityId Source Scalar
Source            ::= "EMPIRICAL" | "DESIGNER_INTENDED" | "INFERRED" | "DEFAULT"
Scalar            ::= Float (0.0 .. 1.0)

Reification       ::= "reify" EdgeId
```

A.3 Relation algebra

Primary relations. Read the table with two questions: can the relation be read backwards (symmetric)? Does it chain (transitive)?

Symbol	Name	Symmetric	Transitive	Meaning
EQL	Equivalence	Yes	Yes	Same referent
ISA	Is-A	No	Yes	Subsumption (A ISA B)
INS	Instance-Of	No	No	Instantiation (a INS A)
PAR	Part-Of	No	Yes	Mereological parthood
CST	Constitutes	No	Yes	Inverse of PAR
PRC	Produced-By	No	Yes	Causal / production dependency
PRE	Precedes	No	Yes	Temporal / logical precedence
LOC	Located-In	No	Yes	Spatial / topical containment
USF	Used-For	No	No	Teleological function
ANT	Antonym	Yes	No	Semantic opposition
PRP	Has-Property	No	No	Attribute attachment
OTH	Other	Yes	No	Fallback for unclassified

Meronymy subtypes refine PAR and CST. Each one says *how* a part belongs to a whole:

Symbol	Expansion	Meaning
CIN	Component-Integral	Essential functional component
MEM	Member-Collection	Interchangeable member of a collection
POR	Portion-Mass	Measured portion of a mass or substance
STU	Stuff-Object	Material constituting an object
FEA	Feature-Activity	Feature or activity of a process
PLA	Place-Area	Sub-area within a larger area

Associative labels are plain-language refinements.
HAS_PROPERTY refines PRP; USED_FOR refines USF;
LOCATED_IN refines LOC; PART_OF refines PAR;
IS_A refines ISA; and OTHER is the fallback.

A.4 Epistemic layer

```
EpistemicTag      ::= NegTag | ConfidenceTag | SalienceTag | FacetTag
NegTag            ::= "NEG" ("0" | "1")           -- 0 = negated, 1 = asserted
ConfidenceTag     ::= "CNF" ("0" | "1" | "2" | "3")
                      -- 0=certain, 1=likely, 2=plausible, 3=unknown
SalienceTag       ::= "SAL" Float                 -- 0.0 .. 1.0
FacetTag          ::= "FAC" FacetId
ReificationTag    ::= "REI" EdgeId                -- reify edge as node
```

Rules: NEG = 0 on an ISA edge means classical negation ("A is not a B"). CNF = 0 means deductively certain; CNF = 3 means unknown. SAL at or above 0.5 means "include by default."

FAC partitions the graph into overlapping points of view. REI lets an edge become a node, so that statements can be made about statements.

A.5 Tiered loading control

```
LoadPolicy        ::= ResidentPolicy | PointedPolicy | CondensedPolicy
ResidentPolicy    ::= "LD" GraphHandle "resident"  -- stays in Tier-1 context
PointedPolicy     ::= "NDL" GraphHandle "pointed"
                      -- load on demand, evict after use
CondensedPolicy   ::= "RSP" GraphHandle "condensed"
                      -- summary only (stats + handles)

LoadDecision      ::= "load" GraphHandle LoadPolicy
EvictDecision     ::= "unload" GraphHandle
GapDecision       ::= "gap" GraphHandle ResidentHandle  -- diff vs resident
ProveDecision     ::= "prove" GraphHandle GraphHandle
                      -- structural equivalence
```

Invariant (MSCA). The Tier-1 resident budget is at most 640 tokens (p50k_base). Tier-2 sections are loaded by phase and tag with `python -m msca.loader -phase N -tag T`. A non-zero exit means failure: never continue from a partial load.

A.6 Encoding grammar (POINTS glyph)

```
Glyph             ::= Header Content Parity
Header            ::= Magic Version Sequence ParityInfo EntityInfo EdgeInfo
                      PayloadInfo GraphId RegionOffsets
Magic             ::= "PTS\x01" (4 bytes)
Version           ::= UInt8 (1)
Sequence          ::= SeqId SeqIndex SeqTotal
SeqId             ::= UInt32
SeqIndex          ::= UInt16
SeqTotal          ::= UInt16
ParityInfo        ::= ParityRatio ParityAlgoId
ParityRatio       ::= UInt8 (0..100)
ParityAlgoId      ::= UInt8 (1=XOR, 2=RS)
EntityInfo        ::= EntityCount EntityIndexBits ConfidenceBits
EntityCount       ::= UInt16 (0..65535)
EntityIndexBits   ::= UInt8 (1..16)
ConfidenceBits    ::= UInt8 (2 fixed)
EdgeInfo          ::= EdgeCount
EdgeCount         ::= UInt16 (0..65535)
PayloadInfo       ::= FacetCount SalienceCount LiteralPoolBytes
GraphId           ::= UInt32
RegionOffsets     ::= Offset[7] (UInt24 each)

Content           ::= EntityTable EdgeTable Salience Facets Reification
                      LabelPool LiteralPool
EntityTable       ::= EntityRecord* LabelPool
EntityRecord      ::= Flags CanonicalId? LabelOffset LabelLen PropertiesLiteral?
Flags             ::= UInt8 (bit0=label, bit1=canonical_id,
                             bit2=properties_literal)
```

```
CanonicalId       ::= Bytes[8] (UTF-8, padded)
LabelOffset       ::= UInt16
LabelLen          ::= UInt8
PropertiesLiteral ::= PropCount (KeyLen Key LiteralValueBytes)*
PropCount         ::= UInt8

EdgeTable         ::= EdgeRecord*
EdgeRecord        ::= CompactEdge | FullEdge
CompactEdge       ::= "0" Source(EIB) "0" Target(EIB)
                      RelationType(3) Polarity(1)
FullEdge          ::= "1" Source(EIB) TargetKind TargetBits RelationType(3)
                      Subtype(3) Confidence(2) Polarity(1) Reified(1)
TargetKind        ::= "0" (entity) | "1" (literal)
TargetBits        ::= max(EIB, 16)

Parity            ::= XORParity | RSParity
XORParity         ::= Bytes[parity_size] (ratio% of content)
RSParity          ::= RS(255,223) parity bytes (32 bytes per 223-byte block)
```

Constants. GLYPH_SIZE = 8192 bytes; HEADER_SIZE = 42 bytes; CONTENT_SIZE = 6920 bytes; default parity ratio 15% (XOR) or 32 parity bytes per block (RS). Maximum entities per glyph: about 460 in a simple chain, about 300 with rich edges. Larger graphs use multi-glyph chaining via sequence numbers.

A.7 Semantic capacity metric (formal)

```
SC(R, G) = |sem(G)| / |R|  (bits-of-meaning per byte)

|sem(G)| = sum over p in propositions(G) of w(p) * bits(p)
  w(p) = 1 (uniform weight)
  bits(p) = 10 (conservative estimate per atomic proposition)

propositions(G) = { entity_label(e) | e in E }
                union { entity_cid(e) | e in E and cid(e) != empty }
                union { edge(e) | e in Edges }  -- 6-field tuple
                union { facet(f) | f in Facets }
                union { salience(s) | s in Salience }
```

Baselines. Compact JSON, Markdown, YAML, token stream (BPE), and random bytes. *Corruption conditions:* clean round trip; RS repair at 1, 4, 8, 16, and 32 damaged bytes per 223-byte block; truncation to 10%, 25%, and 50%; adversarial reordering, synonym substitution, and removal of epistemic tags.

A.8 Optimality scope (honest)

O-OIS is a regulative ideal (in Kant's sense), not a provable global optimum. The claims made are: structural completeness (every known failure mode of predecessor codecs is addressed, patches B1–B4); comparative design quality (the best characterised alternative for maximising semantic capacity under fixed-size, error-correcting, human-checkable constraints, as of November 2026); and indexicality (every claim is bounded by context, date, and knowledge horizon, expiring 22 December 2026).

A.9 Disclosure block (per MSCA)

```
LOADED     <G:points> <G:msca> <G:65536>
NOT LOADED (all further handles: outside this paper's scope)
```

Verification. This grammar was checked against the POINTS example set (7 entries), a capacity sweep (1 to 460 entities), and a Reed–Solomon damage test (0 to 32 damaged bytes).

B Reproducibility Checklist

Appendix: Reproducibility Checklist

This checklist follows the ML Reproducibility Checklist (Pineau et al.) and the ACM Artifact Review criteria, extended for systems research with executable verification checks.

For All Experiments

- **Code availability**: All source code in GitHub repositories (see §10 for URLs).
- **License**: MIT (all code); paper text: CC-BY-4.0.
- **Dependencies**: none beyond a stock Python $\geq$ 3.9 interpreter for the listed checks (the POINTS test suite additionally needs the `pytest` package). The 65536 hierarchy model has an optional Julia companion (`hierarchy.jl`).
- **Hardware**: commodity desktop hardware (AMD Ryzen 9 reference machine, 2026). No specialised hardware required.
- **Runtime**: POINTS full test suite $\approx$ 2.4 s; the MSCA self-checks complete in seconds on the same machine. Wall-clock varies with hardware; the verdicts do not.
- **Random seeds**: the codec measurements and the verification checks are deterministic by construction — the run-twice-diff-zero check (`make check`) fails on any run-to-run difference. The agent-context estimate (Table 8) goes through the model path and is treated as partially reproducible (see below).

POINTS Reproduction

1. Clone: `git clone https://github.com/ttx89-dev/POINTS`
2. Install: not required — Python 3.9+ with `pytest` available is sufficient.
3. Run tests: `python -m pytest suite/tests/ -v`
4. Expected: 94 tests + 42 subtests PASS (2.4 s on the reference machine, 2026-11-04; wall-clock varies by hardware).
5. Golden corpus:
   `python -c "import suite.tests.golden_corpus as g; print(len(g.CORPUS))"` $\rightarrow$ 7 entries.
6. Capacity sweep and RS recovery: the measurements behind Tables 4 and 6 are recorded, with section tags, in `supplements/raw_data.csv`; they were produced by applying the `suite` codec modules to the sweep graphs. No separate sweep scripts are shipped with the repository.

7. Red-team: `python -m pytest suite/tests/test_redteam.py -v` (14 patterns).
8. Run-twice-diff-zero: `make check` (runs each verification command twice and requires identical, timing-normalised output).

MSCA Reproduction

1. Clone: `git clone https://github.com/ttx89-dev/MSCA`
2. Checks: `python -m msca.gates`
3. Expected: 15/15 PASS + corruption check (3 injected defects detected).
4. Capacity verify: `python -m msca.capacity -verify` → MATCHES code.
5. Summary: `python -m msca -summary` → resident 621 tok, budget 640.
6. Loader: `python -m msca.loader -phase 1` (and phases 2–15).
7. Coverage: `python -m msca.loader -coverage` → 0 unreachable.

Measured Numbers Reproduction

All numbers in Tables 3, 4, 5, 6, 7, 8 can be regenerated by:

```
cd arxiv_submission
python scripts/export_raw_data.py > supplements/raw_data.csv
python scripts/generate_figures.py
```

The CSV contains every measured value with section tags. Figures are generated from the same data. These scripts reference the reference machine's repository locations; on a fresh machine, the repository-level commands above reproduce the verdicts without path setup, and `supplements/raw_data.csv` ships with the paper source.

Verification Suite Execution

The MSCA checks are executable, and their corruption check verifies both the checks and their failure detection:

```
python -m msca.gates
# -> ALL GATES PASS  (15/15 gates, self_test=pass)
```

The corruption check injects three defects (a resident-budget overflow, a false self-certification, decorative padding) and must detect all of them. The run-twice audit is `make check`: each verification command executes twice and the outputs must match once wall-clock timings are normalised.

Honesty Checklist (MSCA Invariants)

1. Verified = executed NOW in this session (not inherited, not assumed).
2. Flags are not proof — payload verified, not status field.
3. No completion claim without completion evidence.
4. Tier reported: Tier 3 answering is legitimate; implying Tier 1 is not.
5. Unresolved is a valid result — preferred over plausible fabrication.
6. No self-certification — claim what executed; let evidence speak.
7. When pressure and honesty conflict — honesty wins and stated.

Known Non-Reproducible Elements

- **Agent self-report SC** (Table 8): Estimated via the agent's own self-report, not an independent coder. Inter-rater κ not measured.
- **Wall-clock times**: Vary by hardware. Logical results (pass/fail, SC ratios) are hardware-independent.

Contact

Questions: open an issue on the respective repository.